

PSU Dealing

LINE OA chatbot ภาครัฐ ด้วย Hybrid RAG (Dense + Graph)
ทำไปถึงไหน ทำงานยังไง และเหลืออะไร

44

chunk ความรู้
จากงานวิจัย

300

ผู้ใช้จำลอง
+ 432 events

69

รหัสใน
taxonomy กลาง

5/8

หัวข้อตามเกณฑ์
มีโค้ดแล้ว

อัปเดต 26 กันยายน 2026 · อ้างอิงเกณฑ์จาก **score.md** และแผนงานจาก **req.md**

ข้อมูลจาก branch **main** (รวม Dense ของฟาริกแล้ว) + branch **dev/graph** (Graph ของบังเมษ) +
งานที่ยังไม่ commit ในเครื่อง

ผู้อ่าน: พิน (Data & Knowledge Base) และทิม

1

ภาพรวมสถานะตามเกณฑ์ให้คะแนน

เกณฑ์ score.md ให้คะแนนตาม "คุณภาพการนำไปใช้จริง + หลักฐานจากการทดลอง" ไม่ใช่แค่มีเทคโนโลยี ตารางนี้ประเมินจากโค้ดและผลที่รันได้จริง ณ วันนี้

หัวข้อ (คะแนน)	ผู้รับผิดชอบ	สถานะ	ความคืบหน้า	สรุปสั้น
1. Data & Knowledge Base (10)	พีท	ทำแล้วเกือบครบ		taxonomy, mock, ingest, policy รูปลักษณ์ ครบ · ขาด OCR หนังสือ + ข้อมูลเชิงจริง
2. Dense RAG (15)	ฟาริก	ทำแล้ว ต้องเชื่อม		bge-m3 + จัดคู่สองทาง + penalty + ทดลอง grid · ประเมินบน mock ชุดเก่า ยังไม่คืนค่าตามสัญญาณกลาง
3. Graph RAG (15)	บังเมษ	สร้างกราฟแล้ว		Neo4j 406 nodes / 6,619 edges + ทดสอบ 13 ข้อ · ยังไม่มี retrieval, อยู่คนละ branch, พังกับ taxonomy ใหม่
4. Hybrid RAG (20) ★	—	ยังไม่เริ่ม		มีแค่ stub RRF สำหรับทดสอบ · หัวใจของคะแนน ต้องเริ่มทันที
5. Local LLM (รวม 6 = 15)	พีท	ทำแล้ว		Ollama 4 งาน + guards + benchmark 7 โมเดล วัด เวลา/RAM/GPU · ต้องรันซ้ำเมื่อ Dense/Graph จริงพร้อม
6. API LLM	—	ยังไม่เริ่ม		ยังไม่มีเรียก API จริง ไม่มีการวัด cost/token
7. System Integration (10)	—	ยังไม่เริ่ม		ออกแบบ LINE webhook แล้ว แต่ยัง ไม่มีโค้ด · มีสัญญาณกลาง retrieval แล้ว
8. Evaluation (10)	ทุกคน	บางส่วน		มี ground truth + metric ของ Dense และ LLM · ยังไม่มีตาราง Dense vs Graph vs Hybrid ที่ใช้ของ จริง

มีโค้ดและผลรันจริง มีโค้ดแต่ยังไม่ครบหรือยังไม่เชื่อมกัน ยังไม่มีโค้ด

3 เรื่องที่ต้องรู้ก่อนอ่านต่อ

① Hybrid RAG (20 คะแนน) ยังไม่เริ่ม

เกณฑ์เขียนไว้ว่า "หัวข้อนี้ควรเป็นหัวใจสำคัญของการประเมิน" ตอนนี้มีแค่ตัวรวมผลชั่วคราว (RRF) ที่ผมเขียนไว้ทดสอบ
ฝั่ง LLM ยังไม่มีการ fusion/penalty/routing ที่จริง และยังไม่แสดงผลทดลองเทียบกับ Dense-only / Graph-only

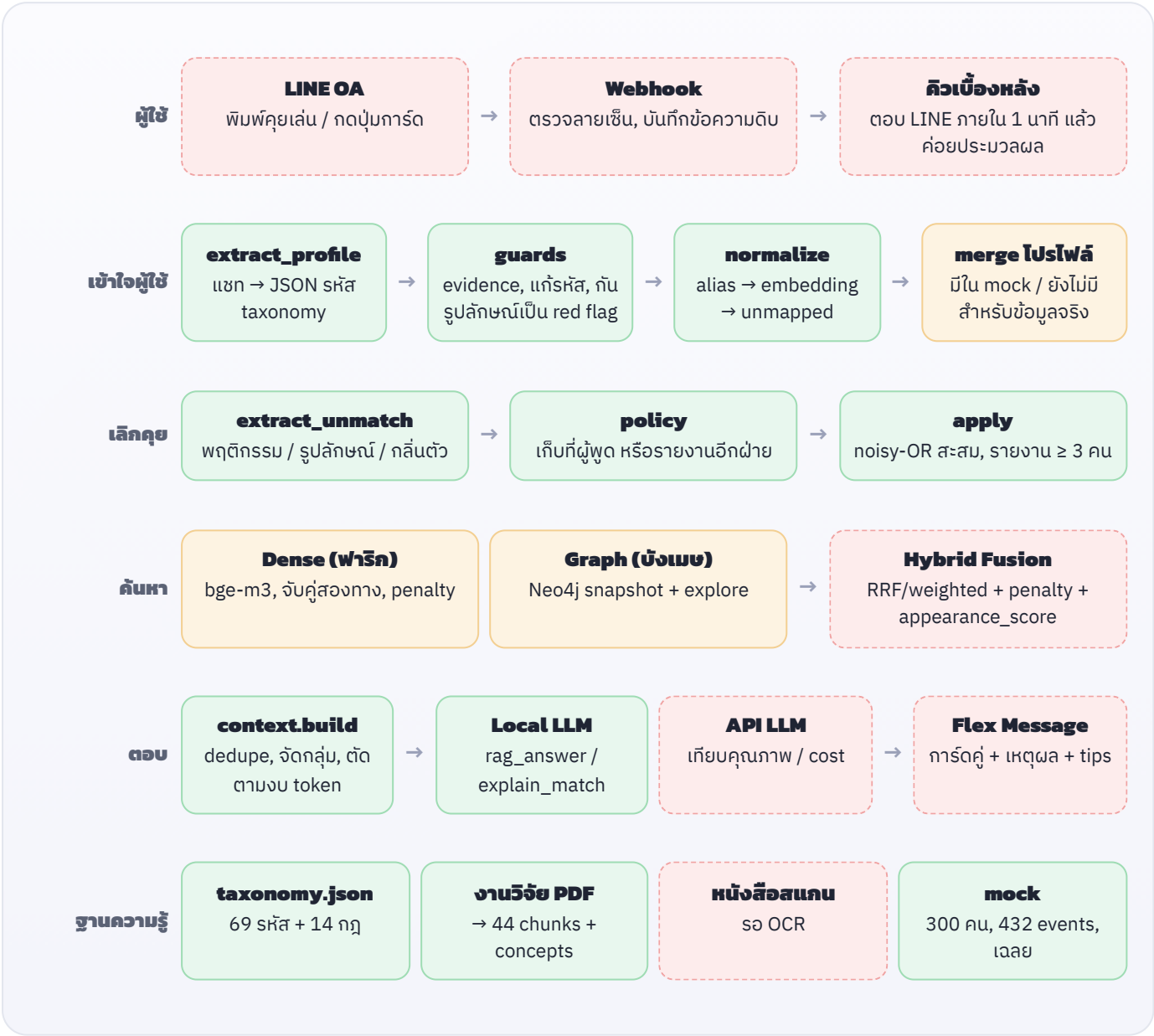
② ข้อมูลกับโหนดของแต่ละคนยังไม่ตรงกัน

mock data ถูกสร้างใหม่ (เพิ่มรูปลักษณ์) หลังจากที่ได้ Dense และ Graph ถูกพัฒนา — **Graph build** กับข้อมูลชุดใหม่ไม่ผ่าน (ทดสอบแล้วขึ้น `Dangling relationship`) และผลทดลองของ Dense เป็นของข้อมูลชุดเก่า (292 คนที่ยินยอม → ชุดใหม่ 286 คน)

③ แต่ละส่วนยังคุยกันผ่าน "สัญญากลาง" ไม่ได้

ผมกำหนด `RetrievalResult` ไว้ใน `pipelines/retrieval/contract.py` ให้ Dense / Graph / Hybrid คืนค่ารูปแบบเดียวกัน แล้ว Local LLM ใช้ได้ทันที — Dense ของฟาริกยังคืนค่าเป็น dict ของตัวเอง และเรียก LLM เองผ่าน `dense/rag.py` , Graph ยังไม่มีฟังก์ชัน `retrieve`

ภาพนี้คือระบบเป้าหมายตาม req.md ระบายสีตามสถานะจริง — เขียว = มีโค้ดใช้งานได้, เหลือง = มีแต่ยังไม่เชื่อม, แดงเส้นประ = ยังไม่มี



ข้อมูลไหลผ่านระบบอย่างไร (ตัวอย่างบันทึกจาก req.md)

จังหวะ	เกิดอะไรขึ้นในระบบ	สถานะ
บันทึก: "ชอบฟังเพลง ซัล ทำกับข้าวกินเอง ชอบคนใจเย็น"	LLM สกัดเป็น hobby:lofi_music , hobby:cooking , wants: trait:calm พร้อม evidence → ตรวจสอบว่า evidence มีในแซทจริง → สะสม confidence ในโปรไฟล์ → สร้าง persona_text ใหม่ → re-embed + อัปเดต edge ใน Graph	สกัด/ตรวจสอบ ✓ merge จริง ✗
บันทึกเล็กคุย: "ติด เพื่อน หายเจียบ"	สกัดเป็น rf:friend_priority , rf:stonewalling → เพิ่มใน avoids ของ บันทึก + นับรายงานให้อีกฝ่าย (ใช้ได้เมื่อ ≥ 3 คน) → ครั้งหน้า Graph กรองออก / Dense หักคะแนนคนคล้ายเดิม	ทำแล้ว
ระบบส่งการ์ดคุณ แพรว	Dense + Graph หาผู้สมัคร → Hybrid รวมคะแนน → ดึง chunk จากหนังสือที่ตรงจุด เสียงของคู่ → LLM เขียนเหตุผลและ tips → ส่ง Flex Message	Hybrid + LINE ✗

หน้าที่: เตรียมข้อมูลทุกชนิดให้ Dense และ Graph ใช้ได้ทันที และให้มีเฉลยสำหรับการวัดผล

3.1 Taxonomy กลาง — "พจนานุกรมที่ทุกส่วนใช้ร่วมกัน"

ผู้ใช้พูดว่า "หายเจียบ", "เจียบใส่", "ปล่อยให้เดา" และหนังสือเขียน "Silent treatment" — ทั้งหมดถูกแปลงเป็นรหัสเดียว `rf:stonewalling` ทำให้ข้อมูลผู้ใช้ หนังสือ และ Graph เชื่อมกันได้

11

หมวด: hobby, trait, comm, attach, love language, love component (Sternberg), red flag, research factor, body, skin, hygiene

69

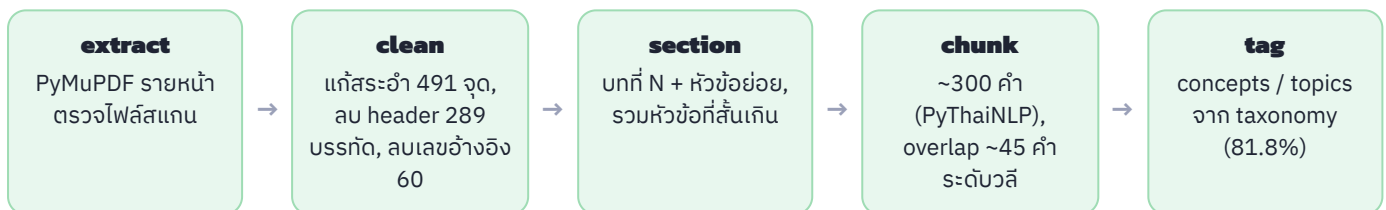
รหัส แต่ละรหัสมี label ไทย + aliases (คำที่คนพูดจริง) + แหล่งอ้างอิง

14

กฎความเข้ากันได้ เช่น anxious $\rightarrow$ avoidant = CONFLICTS_WITH (จงจสไล่-หนี)

ไฟล์: `data/taxonomy.json` (แก้ด้วยมือ) $\rightarrow$ อ่านผ่าน `pipelines/common/taxonomy.py` ที่เดียว $\rightarrow$ ตรวจสอบด้วย `validate_taxonomy.py` (id ซ้ำ, prefix, กฎอ้างอิง id ที่ไม่มี, alias ซกัน) — รอบแรกเจอ "ซีเหวี่ยง" ซกันระหว่าง trait กับ red flag แก้แล้ว ตอนนี้ 0 errors

3.2 Ingest งานวิจัย PDF $\rightarrow$ Knowledge Chunks



ปัญหาที่เจอจริง	วิธีแก้	ผล
สระอำแยกเป็น "ส ศัญ"	regex พยัญชนะ + ช่องว่าง + ำ $\rightarrow$ ำ (ภาษาไทยไม่มีคำขึ้นต้นด้วย ำ)	เหลือ 0 จุด
บรรทัดเนื้อหาหลงท้าย "(English)" ถูกจับเป็นหัวข้อ	หัวข้อต้องขึ้นย่อหน้าใหม่	23 $\rightarrow$ 19 หัวข้อถูกต้อง
หัวข้อสั้นได้ chunk แค่ 15 คำ	รวมหัวข้อ < 80 คำเข้ากับหัวข้อถัดไป	chunk เล็กสุด 96 คำ
overlap ยกทั้งย่อหน้ามาซ้ำ	ยก overlap ระดับวลี	ซ้อนกัน ~15% ตามตั้งใจ

เก็บเฉพาะบทคัดย่อ บท 1, 2, 5 (บท 3 = วิธีวิจัย, บท 4 = ตารางสถิติ ไม่ช่วยตอบคำถาม) · ผลลัพธ์:

`data/processed/book_chunks.jsonl`, `ingest_report.json` (มี before/after ไว้ใส่รายงาน)

3.3 Mock Dataset — ผู้ใช้จำลองที่มีเฉลย

ไฟล์	จำนวน	ใช้ทำอะไร
users.json	300	โปรไฟล์จาก 6 archetype ให้บุคคลทดสอบคล้ายกัน · มี summaries พร้อม embed · ข้อมูล red flag / รูปลักษณะจริงไว้ใน field _ground_truth_*
events.jsonl	432	unmatch 162 (เหตุการณ์ผิดกฎหมาย 143 / รูปลักษณะ 25) · matched 72 · pass 198
ground_truth_pairs.json	286	คู่ที่ควรแมตช์ top-5 ต่อคน + must_exclude เฉลี่ย 13.7 คน (มี red flag ที่เราเสี่ยง)
chats.jsonl	100	แชทจำลอง + เฉลยการสกด มี noise (555, emoji) และเบอร์โทร 11 แชท

ยังไม่ได้ทำ: ① OCR หนังสือ "ใครไม่รักช่างแม่ง" (27 หน้า ไฟล์สแกน — โค้ด `ocr.py` พร้อม ขาดแค่ติดตั้ง tesseract หรือใช้ qwen2.5vl ที่มีในเครื่อง) ② merge โปรไฟล์จากแชทจริง + decay + เพดานขนาด (ออกแบบแล้ว) ③ ชุดทดสอบที่คนอื่นเขียนโดยไม่ดู aliases

4

Feedback & Policy · เลิกคุยแล้วระบบเรียนรู้อะไร

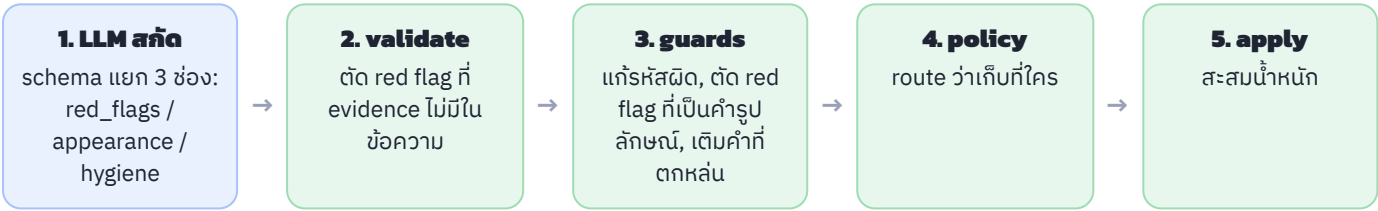
ทำแล้ว

หลักที่ทีมตกลงกัน: รสนิยม (รวมรูปร่างและสีผิว) เป็นสิทธิ์ของผู้ใช้ ใช้นวก/ลบคะแนนได้ — แต่ข้อมูลรูปลักษณะของผู้สมัครต้องมาจากเจ้าตัวบอเอง และผู้ถูกพูดถึงจะไม่ถูกบันทึกอะไรจากคำบ่นของคนอื่น

4.1 กติกา (เขียนเป็นโค้ดใน pipelines/feedback/policy.py)

หมวด	ตัวอย่าง	เก็บที่ผู้พูด (A)	เก็บที่ผู้ถูกพูดถึง (B)	ใช้กับคะแนน
พฤติกรรม	หายเจียบ, หึงหวง, โทหก	avoids	reported_traits (≥ 3 คน)	หักคะแนน / กรองออก
รูปร่าง	อ้วน, ผอม, ลำ	avoids / wants	ไม่เก็บ	soft score × w_appearance (0.2)
สีผิว	ขาว, ผิวดำ	avoids / wants	ไม่เก็บ	soft score · ผู้สมัครต้องยินยอม (PDPA ม.26)
สุขอนามัย	กลิ่นตัว	wants: ดูแลตัวเอง	ไม่เก็บ	soft score

4.2 เสรจริง: "อยากเลิกคุยเพราะอ้วนไป ดำด้วย กลิ่นตัวแรงไป"



สิ่งที่ LLM ทำผิดจริง (ทดสอบกับโมเดลจริง)	ด้านที่จับได้
Typhoon 3B แต่ง red flag 4 รายการ เช่น "พูดจาไม่ให้เกียรติ" ที่ไม่มีในประโยค	validate: evidence ไม่อยู่ในข้อความ → ตัดทิ้งทั้ง 4
ทั้ง Typhoon และ Qwen จัด "อ้วน" เป็น body:athletic / body:average	guard ใหม่: keyword ชื่อว่า "อ้วน" = body:curvy → แก้รหัส
"หายเจียบ" ถูกจัดเป็น ghosting, "ขี้หึง" เป็น "ติดเพื่อน"	guard: evidence ตรง alias ของ red flag ตัวอื่น → แก้รหัส

ผลสุดท้าย: A → avoids body:curvy , skin:dark + wants hygiene:self_care · B → ไม่ถูกบันทึกอะไรเลย · A จะไม่เจอ B อีก (event unmatched)

4.3 ตัวตรวจคำรูปลักษณ์ (feedback/sensitive.py)

คำยาว ("ผิวขาว", "หุ่นดี") จับได้ทันที · คำสั้นกำกวม ("ขาว", "ดำ") ต้องอยู่ที่ขอบคำ ไม่ติดคำสิ่งของ และในแชทปกติต้องมีคำบ่งว่าพูดถึงคน → "กินข้าวขาว" และ "ถ่ายรูปขาวดำ" ไม่ถูกจับ

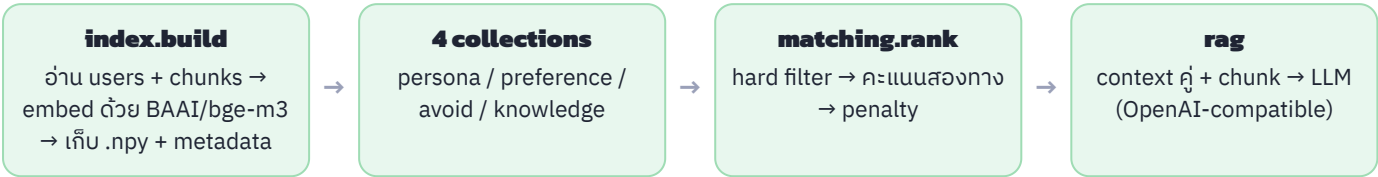
Benchmark: เหตุผลเล็กน้อยที่ป็นรูปลักษณ์ 20 ข้อ	F1	false red flag	p50
Qwen2.5 7B	0.964	0	0.86 s
Typhoon2 8B	0.929	0	0.80 s
Typhoon2 3B	0.931	1	0.53 s

รูปลักษณ์ไม่เคยกลายเป็น red flag ในทุกโมเดล · false red flag ที่เหลือคือ red flag สับสนกันเอง · ⚠ ชัดกทดสอบเขียนจากคำใน aliases จึงสูงเกินจริง

ออกแบบกันการเรียนรู้แฝง: summaries (ข้อความที่ Dense embed) ถูกกรองรูปลักษณ์ออกโดยเจตนา — ถ้าใส่ไว้ Dense penalty จะ "เรียนรู้" ว่า A ไม่ชอบหน้าตาแบบไหนโดยไม่มีใครตั้งกฎและตรวจสอบไม่ได้ รูปลักษณ์จึงใช้ผ่าน Graph tag + `policy.appearance_score()` เท่านั้น

อยู่ใน branch main แล้ว (commit "Add dense retrieval and evaluation pipeline") โคลด์ใน pipelines/dense/

5.1 การทำงาน



ขั้น	รายละเอียด
Hard filter	ยินยอมทั้งคู่ · เพศตรงกันสองทาง · ไม่ใช่ตัวเอง · ไม่เคย unmatched/pass กัน
คะแนนสองทาง	forward = sim(A.preference, B.persona), reverse = sim(B.preference, A.persona) → harmonic mean (ถ้าฝั่งหนึ่งต่ำ คะแนนรวมต่ำ)
Negative penalty	$\max \text{sim}(\text{B.persona}, \text{persona ของคนที่ A เคย unmatched}) \times \lambda$
Knowledge search	cosine uu chunks + กรอง category / concept ได้
Evaluate	grid Top-K × threshold × λ → Precision/Recall/MRR/nDCG/ Violation@K อ่านเฉลยเฉพาะใน evaluation.py

5.2 ผลทดลองที่ฟาริกรันไว้ (bge-m3 จริง, mock ชุดเก่า)

K	threshold	λ	P@K	R@K	MRR	nDCG	Violation@K
5	0.00	0.00	0.158	0.158	0.349	0.170	0.090
5	0.65	0.00	0.071	0.071	0.159	0.078	0.029
10	0.00	0.15	0.140	0.280	0.373	0.236	0.095
20	0.00	0.00	0.109	0.436	0.384	0.303	0.110

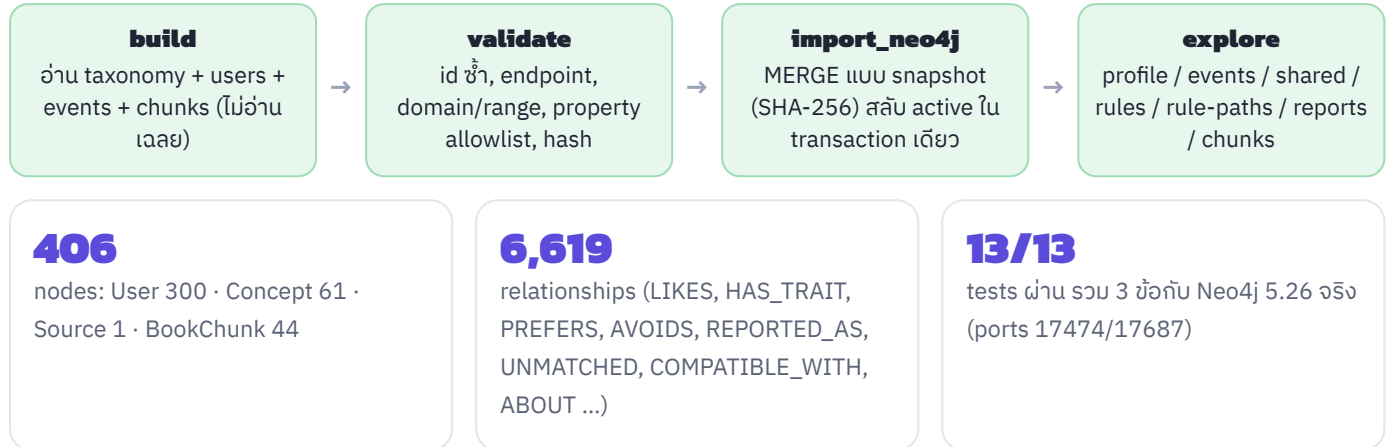
ข้อสรุปของฟาริก (ตรงกับที่ออกแบบไว้): K สูงขึ้น recall ขึ้นแต่ precision ลง · penalty ลด violation ได้เล็กน้อยแต่ไม่ช่วย precision · threshold 0.65 ลด violation เพราะค้นผู้สมัครน้อยลง แต่ recall หายมาก · **Dense อย่างเดียวรับประกันการกรอง red flag ไม่ได้ → ต้องให้ Graph/Hybrid ใช้เงื่อนไขเชิงโครงสร้าง** — นี่คือเหตุผลที่ต้องมี Hybrid พอดี (Violation@K ~0.09–0.11 คือเป้าที่ Hybrid ต้องลดให้ใกล้ 0)

5.3 สิ่งที่ต้องทำต่อ

งาน	ทำไม
build + evaluate ใหม่บน mock ชุดใหม่	ข้อมูลเปลี่ยน (286 คนยินยอม, มีรูปลักษณ์, events ใหม่) ผลเดิมใช้เทียบกับ Graph/Hybrid ไม่ได้
เพิ่ม adapter คืน RetrievalResult	ให้ Local LLM และ Hybrid ใช้ได้ทันที (kind = candidate / chunk)
ใช้ pipelines/llm/tasks.rag_answer แทน dense/rag.py	ได้ guards, การตรวจ citation, metric เวลา/RAM และเทียบโมเดลได้ในทีเดียว
ลอง Cross-Encoder rerank	req.md สะบู่ไว้ และเกณฑ์ Level 5 พุดถึง reranking

อยู่ใน branch **dev/graph** (ยังไม่ merge เข้า main) โค้ดใน `graph/` · ขอบเขตที่ทำ: สร้างและสำรวจกราฟ ยังไม่ทำ retrieval / ranking / RAG

6.1 การทำงาน



จุดที่ทำได้ดีมาก (ใช้ตอบกรรมการได้)

- **property allowlist** — ห้ามส่ง `_ground_truth_*`, `gold_extracted`, `raw_reason` เข้ากราฟ (กัน data leakage และไม่เก็บคำบ่นดิบ)
- **REPORTED_AS** เฉพาะ **usable** และ ≥ 3 และติด `assertion = reported_not_verified`
- กฎความเข้ากันได้ติด `unverified_taxonomy_rule` · ABOUT ติด `keyword_tag_not_entailment` — แยก "สิ่งที่เอกสารพูด" ออกจาก "ข้อสรุปที่ยังไม่ตรวจ"
- snapshot ทำให้ import ซ้ำไม่เพิ่ม node และย้อนดูเวอร์ชันเก่าได้ (audit)

6.2 ปัญหาที่ตรวจพบ

Graph build ไม่ผ่านกับ mock ชุดใหม่

ทดสอบใน worktree ชั่วคราว: ข้อมูลชุดเก่า build ผ่าน · ชุดใหม่ขึ้น `ValueError: Dangling relationship` เพราะ `schema.GROUP_LABELS` รู้จักแค่ 8 หมวดเดิม แต่ผู้ใช้นี้มี `PREFERS/AVOIDS` → `body:*` / `skin:*` / `hygiene:*` ที่ยังไม่มี node

แก้: เพิ่ม `body_types` → `BodyType`, `skin_tones` → `SkinTone`, `hygiene` → `Hygiene` ใน `GROUP_LABELS` · เพิ่ม relation `SELF_DESCRIBED` (User → `BodyType/SkinTone`) จาก `appearance.self_described` · ขยาย range ของ `PREFERS/AVOIDS` · ห้ามสร้าง `REPORTED_AS` ไปยังรูปลักษณ์

6.3 สิ่งที่ต้องทำต่อ (ส่วนที่ได้คะแนน Graph RAG จริง)

งาน	รายละเอียด
GraphRetriever คืน RetrievalResult	แปลง path เป็น graph_fact ที่อ่านรู้เรื่อง เช่น "(วิตกกังวล) -[CONFLICTS_WITH]-> (หลีกเลี่ยง): วงจรไล่-หนี"
หาผู้สมัครด้วย Cypher	hard constraint (AVOIDS n REPORTED_AS) + multi-hop theory score (HAS_TRAIT – COMPATIBLE_WITH – HAS_TRAIT) + shared hobby/love language
evaluate ด้วยเฉลี่ยเดียวกับ Dense	P@K / R@K / MRR / nDCG / Violation@K – ให้เทียบในตารางเดียวกันได้
แสดงเคสที่ Graph ชนะ Dense	persona คล้ายกันมากแต่มี red flag ที่ถูกรายงาน · attachment ที่ขัดกันซึ่งข้อความไม่บอตรงๆ
merge dev/graph → main	หลังแก้ schema รูปลักษณ์ และรัน tests ผ่านกับข้อมูลชุดใหม่

Ollama บน Apple M5 Pro · unified memory 24 GB · โค้ดใน pipelines/llm/ (ยังไม่ commit)

7.1 หน้าที 2 แบบ และสัญญากลาง

ป้อนข้อมูลเข้า Dense / Graph

`extract_profile` แยก → JSON schema taxonomy

`extract_unmatch` เหตุผลเล็กน้อย → 3 ข้อ + severity

JSON schema ใช้ schema เป็น enum → นอก taxonomy ไม่ได้

ตอบจาก context ของ Dense / Graph / Hybrid

`rag_answer` ตอบคำถามพร้อมอ้างอิง [n] หรือ "ไม่มีข้อมูลเพียงพอ"

`explain_match` อธิบายเหตุผลจับคู่ 3 ส่วน

รับ `RetrievalResult` จาก retriever ไหนก็ได้

```
# pipelines/retrieval/contract.py - ส่วน 2/3/4 ต้องคืนแบบนี้
RetrievalItem(id, kind="chunk" | "candidate" | "graph_fact", text, score,
source="dense"|"graph"|"hybrid", meta)
RetrievalResult(mode, query, items, latency_ms)
# แล้วใช้ได้ทันที
tasks.rag_answer(query, my_retriever.retrieve(query))
```

7.2 Benchmark งานสกัดโปรไฟล์ (6 โมเดล * 4 config * 25 แชน)

โมเดล	schema+few	json+few	schema+zero	json+zero	p50 (s)	GPU (GB)
Qwen2.5 7B	0.828	0.828	0.669	0.000	2.09	4.62
Typhoon2 8B	0.725	0.707	0.630	0.000	2.09	5.29
Gemma3 4B	0.685	0.602	0.398	0.000	1.40	3.72
Typhoon2 3B	0.676	0.679	0.298	0.000	1.08	2.31
Qwen2.5 3B	0.574	0.569	0.428	0.000	1.23	2.08
Qwen2.5 1.5B	0.390	0.414	0.310	0.000	0.83	1.10

วิเคราะห์: few-shot คือปัจจัยหลัก — json+few ≈ schema+few · schema ช่วยได้เฉพาะตอนไม่มีตัวอย่าง · json+zero ได้ 0 ทุกโมเดลเพราะตอบผิดรูปแบบ (Typhoon ห่อด้วย "field", Qwen ตอบ "cafe" แทน "hobby:cafe") · โมเดลใหญ่ขึ้น F1 ดีขึ้นแต่ tokens/s ลด (1.5B ≈ 170 tok/s vs 7B ≈ 52 tok/s) ตารางนี้รันก่อนเพิ่ม field รูปลักษณะใน prompt → ต้องรันซ้ำก่อนใส่รายงาน

7.3 Benchmark RAG (4 โมเดล * 3 โหมด * จบ context 2 แบบ * 10 คำถาม)

โมเดล (จบ 1,500)	Dense	Graph	Hybrid	อ้างอิง (hybrid)	ตอบ "ไม่มีข้อมูล" ถูก	p50 (s)
Typhoon2 8B	0.417	0.611	0.824	1.00	1.00	1.40
Typhoon2 3B	0.417	0.611	0.824	0.89	1.00	0.97
Gemma3 4B	0.509	0.611	0.750	0.89	1.00	0.80
Qwen2.5 7B	0.398	0.611	0.750	1.00	0.90	1.62

ตัวเลขใน Dense/Graph/Hybrid = keyword recall · Hybrid ชนะทุกโมเดล · จบ 3,000 token ไม่ได้ดีขึ้นแต่ช้าลง ~1.6 เท่า · ⚠ ใช้ retriever ชั่วคราว (Dense = trigram ไม่ใช่ embedding) และมีแค่ 10 คำถาม → ต้องรันซ้ำด้วย Dense/Graph จริง

7.4 คำที่เลือกใช้ (อิงผลทดลอง)

งาน	โมเดล	config	เหตุผล
สกัดข้อมูล	Qwen2.5 7B	temp 0, schema + few-shot, ctx 2048	F1 สูงสุด, false red flag 0 · Typhoon 3B เป็นตัวสำรองเมื่อ RAM ไม่พอ
ตอบ / อธิบาย	Typhoon2 8B	temp 0.3, context 1,500 token	hybrid recall สูงสุด, อ้างอิงครบ, ปฏิเสธถูก

วัดทุกครั้ง: p50/p95 latency, prompt/gen tokens per second, cold load, RAM สูงสุด, CPU, GPU memory, other_models_loaded (ต้องเป็น 0) · ภาษาไทย 1.64 ตัวอักษร/token (calibrate จริง)

8

ส่วนที่ยังไม่ได้เริ่ม

8.1 Hybrid RAG (20 คะแนน) – สำคัญที่สุด ยังไม่เริ่ม

```
final(A,B) = fuse( dense_score(A,B), graph_score(A,B) ) # RRF หรือ weighted normalization
               - λ_rf · redflag_penalty(A,B)           # Graph: AV0IDS n REPORTED_AS (+
Dense negative sim)
               + w_app · policy.appearance_score(A,B) # soft, เฉพาะข้อมูลที่ B นอกเอง
routing: คำถามข้อเท็จจริง/ความสัมพันธ์ → Graph ก่อน · คำถามเชิงความหมาย → Dense ก่อน
context: ผู้สมัคร + graph_fact + chunk ที่ตรงจุดเสี่ยงของคู่ → context.build → LLM
```

ต้องทดลอง	ค่า	วัดด้วย
วิธี fusion	RRF (k=60) vs weighted α ∈ {0.3, 0.5, 0.7}	P@K, R@K, MRR, nDCG
penalty	λ_rf ∈ {0, 0.15, 0.3} + hard filter on/off	Violation@K → ต้องใกล้ 0 (Dense ได้ ~0.09)
สเปกตรัมลักษณะ	w_appearance ∈ {0, 0.1, 0.2, 0.3}	unmatch rate จำลอง
เทียบ	Dense-only vs Graph-only vs Hybrid	ตารางเดียวกัน เผลยเดียวกัน

8.2 API LLM ยังไม่เริ่ม

- เพิ่ม client (เช่น Claude / Gemini / GPT) ใน pipelines/llm/ ให้ใช้ tasks และ benchmark ชุดเดียวกับ Local → เทียบได้ทันที
- วัดเพิ่ม: token in/out, **cost ต่อ call**, latency, error/retry · เหนือ Level 5 ต้องการ "Response Time และ Cost"
- ใช้กับงานที่ต้องการเหตุผลลึก (explain_match) ตาม req.md — และระวังส่งข้อมูลส่วนตัวออกนอกเครื่อง

8.3 System Integration (LINE OA) ยังไม่เริ่ม

ชิ้นส่วน	สิ่งที่ต้องทำ
Webhook (FastAPI + line-bot-sdk)	ตรวจ X-Line-Signature · รับ message / postback / follow / unfollow · ตอบภายใน replyToken (~1 นาที)
Storage	line_users, raw_messages (ลบหลัง 30 วัน), profiles, events, extraction_log
คิวเบื้องหลัง	สะสมทั้ง session แล้วสกัดทีเดียว · อัปเดต Vector + Graph เมื่อโปรไฟล์เปลี่ยนจริง
Flex Message	การ์ดคู่ + เหตุผล + tips + ปุ่ม "ทำความเข้าใจ / ขอฟ้าน / เลิกคุย"
Consent	ขอยินยอมตอน follow · ขอแยกสำหรับข้อมูลสีผิว · unfollow → หลุดจาก pool
Error handling	LLM timeout → fallback โมเดลเล็ก · DB ล่ม · โปรไฟล์ไม่พอ → ชวนคุยต่อ

8.4 Evaluation รวม บางส่วน

มีแล้ว: เผลยการจับคู่, metric retrieval (Dense), benchmark LLM · **ยังขาด:** ตารางเดียว Dense vs Graph vs Hybrid × Local vs API · RAGAS / LLM-as-a-Judge (Faithfulness, Context Precision/Recall) · วิเคราะห์เคสที่ล้มเหลวว่าทำไมวิธีหนึ่งแพ้



ลำดับงานที่แนะนำ

เรียงตามผลต่อคะแนนและสิ่งที้งานอื่นรออยู่

#	งาน	ใคร	ทำไมต้องก่อน
1	Commit + push งานส่วน 1 และ Local LLM (llm, retrieval, feedback, profile, mock ใหม่, RUN.md)	พีท	ทุกคนต้องใช้ข้อมูลและสัญญาณกลางชุดเดียวกัน
2	แก้ Graph schema ให้รองรับรูปลักษณ์ + merge dev/graph	บังเมษ	ตอนนี้ build กับข้อมูลใหม่ไม่ผ่าน
3	Dense rebuild + evaluate ใหม่ + adapter RetrievalResult	ฟาริก	ผลเดิมเป็นของข้อมูลชุดเก่า
4	GraphRetriever (Cypher หาผู้สมัคร + graph_fact) + evaluate	บังเมษ	คะแนน Graph RAG มาจากส่วนนี้
5	Hybrid: fusion + penalty + appearance_score + routing + ทดลอง grid	ทิม	20 คะแนน — หัวใจของเกณฑ์
6	รัน LLM benchmark ชำ้ด้วย retriever จริง + เพิ่ม API LLM	พีท	ตาราง Local vs API สำหรับ Level 5
7	OCR หนังสือ → ingest → rebuild index/graph	พีท	ตอนนี้ red flag หลายตัวไม่มี chunk อธิบาย
8	LINE webhook + Flex + demo 3 ชิ้นจาก req.md	ทิม	System Integration + ใช้สาริต
9	ตาราง evaluation รวม + RAGAS + วิเคราะห์เคสล้มเหลว	ทิม	Evaluation Level 5 ต้องอธิบายสาเหตุได้

แผนที่ไฟล์

ที่อยู่	หน้าที่	สถานะ
data/taxonomy.json	รหัสกลาง + กฎ + appearance_policy	พร้อม
pipelines/common/	paths, io, taxonomy loader, validate_taxonomy	พร้อม
pipelines/mock/	สร้างผู้ใช้/events/แซก/เฉลย (รวม appearance.py)	พร้อม
pipelines/ingest/	PDF → clean → section → chunk → tag	พร้อม
pipelines/feedback/	sensitive, policy, apply	พร้อม
pipelines/profile/normalize.py	คำอิสระ → รหัส	พร้อม
pipelines/retrieval/	contract + stubs ตรวจสอบ	พร้อม
pipelines/llm/	config, client, resources, schemas, prompts, parsing, guards, context, tasks, bench	พร้อม
pipelines/dense/	index, matching, evaluation, rag (ฟาริก)	ต้องเชื่อม
graph/ (dev/graph)	schema, build, neo4j_store, explore (บั้งเมษ)	ต้องแก้ + merge
hybrid / api llm / line	—	ยังไม่มี

คำสั่งหลัก (รายละเอียดใน RUN.md)

```
.venv/bin/python -m pipelines.common.validate_taxonomy          # ตรวจ taxonomy
.venv/bin/python -m pipelines.mock.run --n 300 --seed 42        # mock data
.venv/bin/python -m pipelines.ingest.run                        # PDF → chunks
.venv/bin/python -m pipelines.dense.run build && ... evaluate    # Dense (ฟาริก)
python3 -m graph.build ; python -m graph.import_neo4j           # Graph (บั้งเมษ, branch dev/graph)
.venv/bin/python -m pipelines.llm.run demo | bench ...          # Local LLM
```

ตัวเลขทั้งหมดในเอกสารนี้มาจากการรันจริงบนเครื่องพัฒนา (Apple M5 Pro, 24 GB) กับข้อมูลจำลอง ไม่ได้วัดคุณภาพการจับคู่ในโลกจริง · benchmark ของ LLM อยู่ใน data/eval/llm_bench/ · ผลของ Dense จาก pipelines/dense/EXPERIMENTS.md · ผลของ Graph จาก graph/VERIFICATION.md (branch dev/graph)